

回高速缓存引起的传送比较少，它允许更多的到内存的带宽用于执行 DMA 的 I/O 设备。此外，越往层次结构下面走，传送时间增加，减少传送的数量就变得更加重要。一般而言，高速缓存越往下层，越可能使用写回而不是直写。

旁注 高速缓存行、组和块有什么区别？

很容易混淆高速缓存行、组和块之间的区别。让我们回顾一下这些概念，确保概念清晰：

- 块是一个固定大小的信息包，在高速缓存和主存(或下一层高速缓存)之间来回传送。
- 行是高速缓存中的一个容器，存储块以及其他信息(例如有效位和标记位)。
- 组是一个或多个行的集合。直接映射高速缓存中的组只由一行组成。组相联和全相联高速缓存中的组是由多个行组成的。

在直接映射高速缓存中，组和行实际上是等价的。不过，在相联高速缓存中，组和行是很不一样的，这两个词不能互换使用。

因为一行总是存储一个块，术语“行”和“块”通常互换使用。例如，系统专家总是说高速缓存的“行大小”，实际上他们指的是块大小。这样的用法十分普遍，只要你理解块和行之间的区别，它不会造成任何误会。

6.5 编写高速缓存友好的代码

在 6.2 节中，我们介绍了局部性的思想，而且定性地谈了一下什么会具有良好的局部性。明白了高速缓存存储器是如何工作的，我们就能更加准确一些了。局部性比较好的程序更容易有较低的不命中率，而不命中率较低的程序往往比不命中率较高的程序运行得更快。因此，从具有良好局部性的意义上来说，好的程序员总是应该试着去编写高速缓存友好(cache friendly)的代码。下面就是我们用来确保代码高速缓存友好的基本方法。

1) 让最常见的情况运行得快。程序通常把大部分时间都花在少量的核心函数上，而这些函数通常把大部分时间都花在了少量循环上。所以要把注意力集中在核心函数里的循环上，而忽略其他部分。

2) 尽量减小每个循环内部的缓存不命中数量。在其他条件(例如加载和存储的总次数)相同的情况下，不命中率较低的循环运行得更快。

为了看看实际上这是怎么工作的，考虑 6.2 节中的函数 `sumvec`：

```
1  int sumvec(int v[N])
2  {
3      int i, sum = 0;
4
5      for (i = 0; i < N; i++)
6          sum += v[i];
7      return sum;
8  }
```

这个函数高速缓存友好吗？首先，注意对于局部变量 `i` 和 `sum`，循环体有良好的时间局部性。实际上，因为它们都是局部变量，任何合理的优化编译器都会把它们缓存在寄存器文件中，也就是存储器层次结构的最高层中。现在考虑一下对向量 `v` 的步长为 1 的引用。一般而言，如果一个高速缓存的块大小为 B 字节，那么一个步长为 k 的引用模式(这里 k 是以字为单位的)平均每次循环迭代会有 $\min(1, (\text{wordsize} \times k) / B)$ 次缓存不命中。当 $k=1$ 时，它取最小值，所以对 `v` 的步长为 1 的引用确实是高速缓存友好的。例如，假设 `v` 是块对齐的，字为 4 个字节，高速缓存块为 4 个字，而高速缓存初始为空(冷高速缓存)。然

后，无论是什么样的高速缓存结构，对 v 的引用都会得到下面的命中和不命中模式：

$v[i]$	$i=0$	$i=1$	$i=2$	$i=3$	$i=4$	$i=5$	$i=6$	$i=7$
访问顺序，命中[h]或不命中[m]	1 [m]	2 [h]	3 [h]	4 [h]	5 [m]	6 [h]	7 [h]	8 [h]

在这个例子中，对 $v[0]$ 的引用会不命中，而相应的包含 $v[0] \sim v[3]$ 的块会被从内存加载到高速缓存中。因此，接下来三个引用都会命中。对 $v[4]$ 的引用会导致不命中，而一个新的块被加载到高速缓存中，接下来的三个引用都命中，依此类推。总的来说，四个引用中，三个会命中，在这种冷缓存的情况下，这是我们所能做到的最好的情况了。

总之，简单的 `sumvec` 示例说明了两个关于编写高速缓存友好的代码的重要问题：

- 对局部变量的反复引用是好的，因为编译器能够将它们缓存在寄存器文件中（时间局部性）。
- 步长为 1 的引用模式是好的，因为存储器层次结构中所有层次上的缓存都是将数据存储为连续的块（空间局部性）。

在对多维数组进行操作的程序中，空间局部性尤其重要。例如，考虑 6.2 节中的 `sumarrayrows` 函数，它按照行优先顺序对一个二维数组的元素求和：

```

1  int sumarrayrows(int a[M][N])
2  {
3      int i, j, sum = 0;
4
5      for (i = 0; i < M; i++)
6          for (j = 0; j < N; j++)
7              sum += a[i][j];
8      return sum;
9  }
```

由于 C 语言以行优先顺序存储数组，所以这个函数中的内循环有与 `sumvec` 一样好的步长为 1 的访问模式。例如，假设我们对这个高速缓存做与对 `sumvec` 一样的假设。那么对数组 a 的引用会得到下面的命中和不命中模式：

$a[i][j]$	$j=0$	$j=1$	$j=2$	$j=3$	$j=4$	$j=5$	$j=6$	$j=7$
$i=0$	1 [m]	2 [h]	3 [h]	4 [h]	5 [m]	6 [h]	7 [h]	8 [h]
$i=1$	9 [m]	10 [h]	11 [h]	12 [h]	13 [m]	14 [h]	15 [h]	16 [h]
$i=2$	17 [m]	18 [h]	19 [h]	20 [h]	21 [m]	22 [h]	23 [h]	24 [h]
$i=3$	25 [m]	26 [h]	27 [h]	28 [h]	29 [m]	30 [h]	31 [h]	32 [h]

但是如果我们做一个看似无伤大雅的改变——交换循环的次序，看看会发生什么：

```

1  int sumarraycols(int a[M][N])
2  {
3      int i, j, sum = 0;
4
5      for (j = 0; j < N; j++)
6          for (i = 0; i < M; i++)
7              sum += a[i][j];
8      return sum;
9  }
```

在这种情况下，我们是一列一列而不是一行一行地扫描数组的。如果我们够幸运，整个数组都在高速缓存中，那么我们也会有相同的不命中率 $1/4$ 。不过，如果数组比高速缓存要

大(更可能出现这种情况), 那么每次对 $a[i][j]$ 的访问都会不命中!

$a[i][j]$	$j=0$	$j=1$	$j=2$	$j=3$	$j=4$	$j=5$	$j=6$	$j=7$
$i=0$	1 [m]	5 [m]	9 [m]	13 [m]	17 [m]	21 [m]	25 [m]	29 [m]
$i=1$	2 [m]	6 [m]	10 [m]	14 [m]	18 [m]	22 [m]	26 [m]	30 [m]
$i=2$	3 [m]	7 [m]	11 [m]	15 [m]	19 [m]	23 [m]	27 [m]	31 [m]
$i=3$	4 [m]	8 [m]	12 [m]	16 [m]	20 [m]	24 [m]	28 [m]	32 [m]

较高的不命中率对运行时间可以有显著的影响。例如, 在桌面机器上, `sumarrayrows` 运行速度比 `sumarraycols` 快 25 倍。总之, 程序员应该注意他们程序中的局部性, 试着编写利用局部性的程序。

 **练习题 6.17** 在信号处理和科学计算的应用中, 转置矩阵的行和列是一个很重要的问题。从局部性的角度来看, 它也很有趣, 因为它的引用模式既是以行为主(row-wise)的, 也是以列为主(column-wise)的。例如, 考虑下面的转置函数:

```
1  typedef int array[2][2];
2
3  void transpose1(array dst, array src)
4  {
5      int i, j;
6
7      for (i = 0; i < 2; i++) {
8          for (j = 0; j < 2; j++) {
9              dst[j][i] = src[i][j];
10         }
11     }
12 }
```

假设在一台具有如下属性的机器上运行这段代码:

- `sizeof(int)==4`。
 - `src` 数组从地址 0 开始, `dst` 数组从地址 16(十进制)开始。
 - 只有一个 L1 数据高速缓存, 它是直接映射的、直写和写分配的, 块大小为 8 个字节。
 - 这个高速缓存总的大小为 16 个数据字节, 一开始是空的。
 - 对 `src` 和 `dst` 数组的访问分别是读和写不命中的唯一来源。
- A. 对每个 `row` 和 `col`, 指明对 `src[row][col]` 和 `dst[row][col]` 的访问是命中(h)还是不命中(m)。例如, 读 `src[0][0]` 会不命中, 写 `dst[0][0]` 也不命中。

dst数组			src数组		
列0		列1	列0		列1
0行	m		0行	m	
1行			1行		

B. 对于一个大小为 32 数据字节的高速缓存重复这个练习。

 **练习题 6.18** 最近一个很成功的游戏 `SimAquarium` 的核心就是一个紧密循环(tight loop), 它计算 256 个海藻(algae)的平均位置。在一台具有块大小为 16 字节($B=16$)、整个大小为 1024 字节的直接映射数据缓存的机器上测量它的高速缓存性能。定义如下:

```
1  struct algae_position {
2      int x;
3      int y;
4  };
```



```

5
6  struct algae_position grid[16][16];
7  int total_x = 0, total_y = 0;
8  int i, j;

```

还有如下假设：

- `sizeof(int)==4`。
- `grid` 从内存地址 0 开始。
- 这个高速缓存开始时是空的。
- 唯一的内存访问是对数组 `grid` 的元素的访问。变量 `i`、`j`、`total_x` 和 `total_y` 存放在寄存器中。

确定下面代码的高速缓存性能：

```

1      for (i = 0; i < 16; i++) {
2          for (j = 0; j < 16; j++) {
3              total_x += grid[i][j].x;
4          }
5      }
6
7      for (i = 0; i < 16; i++) {
8          for (j = 0; j < 16; j++) {
9              total_y += grid[i][j].y;
10         }
11     }

```

- 读总数是多少？
- 缓存不命中的读总数是多少？
- 不命中率是多少？

 **练习题 6.19** 给定练习题 6.18 的假设，确定下列代码的高速缓存性能：

```

1      for (i = 0; i < 16; i++){
2          for (j = 0; j < 16; j++) {
3              total_x += grid[j][i].x;
4              total_y += grid[j][i].y;
5          }
6      }

```

- 读总数是多少？
- 高速缓存不命中的读总数是多少？
- 不命中率是多少？
- 如果高速缓存有两倍大，那么不命中率会是多少呢？

 **练习题 6.20** 给定练习题 6.18 的假设，确定下列代码的高速缓存性能：

```

1      for (i = 0; i < 16; i++){
2          for (j = 0; j < 16; j++) {
3              total_x += grid[i][j].x;
4              total_y += grid[i][j].y;
5          }
6      }

```

- 读总数是多少？
- 高速缓存不命中的读总数是多少？

- C. 不命中率是多少？
D. 如果高速缓存有两倍大，那么不命中率会是多少呢？

6.6 综合：高速缓存对程序性能的影响

本节通过研究高速缓存对运行在实际机器上的程序的性能影响，综合了我们对存储器层次结构的讨论。

6.6.1 存储器山

一个程序从存储系统中读数据的速率称为读吞吐量(read throughput)，或者有时称为读带宽(read bandwidth)。如果一个程序在 s 秒的时间段内读 n 个字节，那么这段时间内的读吞吐量就等于 n/s ，通常以兆字节每秒(MB/s)为单位。

如果我们要编写一个程序，它从一个紧密程序循环(tight program loop)中发出一系列读请求，那么测量出的读吞吐量能让我们看到对于这个读序列来说的存储系统的性能。图 6-40

```

code/mem/mountain/mountain.c
1  long data[MAXELEMS];      /* The global array we'll be traversing */
2
3  /* test - Iterate over first "elems" elements of array "data" with
4   *      stride of "stride", using 4 x 4 loop unrolling.
5   */
6  int test(int elems, int stride)
7  {
8      long i, sx2 = stride*2, sx3 = stride*3, sx4 = stride*4;
9      long acc0 = 0, acc1 = 0, acc2 = 0, acc3 = 0;
10     long length = elems;
11     long limit = length - sx4;
12
13     /* Combine 4 elements at a time */
14     for (i = 0; i < limit; i += sx4) {
15         acc0 = acc0 + data[i];
16         acc1 = acc1 + data[i+stride];
17         acc2 = acc2 + data[i+sx2];
18         acc3 = acc3 + data[i+sx3];
19     }
20
21     /* Finish any remaining elements */
22     for (; i < length; i+=stride) {
23         acc0 = acc0 + data[i];
24     }
25     return ((acc0 + acc1) + (acc2 + acc3));
26 }
27
28 /* run - Run test(elems, stride) and return read throughput (MB/s).
29 *      "size" is in bytes, "stride" is in array elements, and Mhz is
30 *      CPU clock frequency in Mhz.
31 */
32 double run(int size, int stride, double Mhz)
33 {
34     double cycles;
35     int elems = size / sizeof(double);
36
37     test(elems, stride);          /* Warm up the cache */
38     cycles = fcy2(test, elems, stride, 0); /* Call test(elems,stride) */
39     return (size / stride) / (cycles / Mhz); /* Convert cycles to MB/s */
40 }
code/mem/mountain/mountain.c

```

图 6-40 测量和计算读吞吐量的函数。我们可以通过以不同的 `size`(对应于时间局部性)和 `stride`(对应于空间局部性)的值来调用 `run` 函数，产生某台计算机的存储器山